Travel Genie AI

Agente textual con herramientas externas para planificación de viajes

Elena Ardura, Victoria García, Jimena Monteagudo y Emma Rey

1. Definición del problema

Planificar un viaje hoy en día implica ir saltando entre muchas plataformas distintas: buscadores de vuelos, webs de hoteles, mapas, guías de ocio, etc. Al final, el problema no es que falte información, de hecho todo lo contrario, hay demasiada y está muy dispersa. El usuario tiene que invertir bastante tiempo en reunirla, compararla y comprobar que todo encaja bien entre sí. Esto hace que una tarea que en principio debería ser sencilla se vuelva pesada, poco eficiente y además propensa a errores. Por ejemplo, puede parecer que un vuelo es una muy buena opción por precio, pero luego aterriza en un aeropuerto mal conectado, o que un hotel es barato, pero está demasiado lejos de las zonas de interés.

En este contexto surge **Travel Genie AI**, un agente conversacional capaz de generar una planificación de viaje a partir de una petición en lenguaje natural, como por ejemplo: *"Quiero ir a Roma del 15 al 20 de junio desde Madrid"*. A partir de esta entrada, el sistema produce una propuesta que integra distintas partes del viaje, incluyendo **(1)** una opción de vuelo adecuada, **(2)** un alojamiento recomendado, **(3)** una alternativa de transporte entre el aeropuerto y el hotel, **(4)** una selección de puntos de interés cercanos y **(5)** una referencia adicional de gastronomía local.

La motivación principal del proyecto es que tiene una utilidad real. Actualmente, muchas consultas relacionadas con la planificación de viajes siguen resolviéndose de forma casi manual, aunque gran parte del proceso consiste en tareas repetitivas como buscar, comparar y combinar opciones. Un agente como este permite reducir mucho ese esfuerzo y ofrecer al usuario una experiencia más cómoda, rápida y personalizada. Además, no se limita a enseñar resultados aislados, sino que intenta construir una propuesta coherente en conjunto. Es decir, no solo busca un vuelo o un hotel por separado, sino que tiene en cuenta si las distintas partes del viaje son compatibles entre sí y si la solución final tiene sentido como plan completo.

1.1. Arquitectura general

El sistema sigue el patrón ReAct (*Reasoning + Acting*) con bucle *Thought* → *Action* → *Observation*, desplegado como servicio HTTP con `FastAPI` y conectado a una interfaz web conversacional. De forma simplificada, el flujo es:

`Usuario → Frontend → FastAPI → TravelAgent (ReAct) → tools/APIs → respuesta final`

La clase `TravelAgent` (`backend/travel_agent.py`) encapsula el bucle principal. El *system prompt* se construye a partir del registro de herramientas y obliga al modelo a seguir el formato:

```
Thought: <razonamiento interno>
Action: <nombre_exacto_de_la_tool>
Action Input: <JSON valido>
Observation: <resultado de la tool>
...
Final Answer: <respuesta al usuario>
```

El backend expone dos modos de interacción: una respuesta estándar mediante `/chat` y un modo en *streaming* mediante `/chat/stream`, que permite enviar la traza del agente en tiempo real a la interfaz. El bucle itera hasta producir `Final Answer` o alcanzar un máximo de `max_iterations = 12`. En cada iteración, la observación generada por la tool se reintroduce en el historial para condicionar el siguiente paso del modelo. De este modo, el agente puede encadenar búsquedas de vuelos, hoteles, transporte, puntos de interés y gastronomía local dentro de una misma planificación.

1.2. Componentes

El repositorio se organiza en los siguientes módulos:

- `src/`: contiene el *frontend* de la aplicación, incluyendo la interfaz conversacional y el panel lateral donde se muestra la traza del agente.
- `backend/api.py`: punto de entrada `FastAPI`, con los *endpoints* `GET /health`, `POST /chat` y `POST /chat/stream`.
- `backend/travel_agent.py`: implementa la clase `TravelAgent`, que contiene el bucle ReAct, el parser de respuestas, la ejecución de tools y la integración con el LLM mediante `transformers`.
- `backend/tools/`: agrupa las integraciones externas del sistema: `flights.py`, `hotels.py`, `transport.py`, `places_of_interest.py` y `food.py`.
- `backend/.env`: almacena las claves y parámetros de configuración necesarios para las APIs externas.

1.3. Registro de *tools*

Cada herramienta se define mediante un `dataclass Tool`, donde se almacenan su nombre, una descripción, los parámetros esperados y la función Python que la ejecuta. A partir de este registro se construye automáticamente el *system prompt*, de modo que añadir una nueva *tool* resulta directo, ya que basta con incorporarla a la lista `TOOLS`.

La ejecución se centraliza en el *dispatcher* `execute_tool`, que se encarga de localizar la herramienta correspondiente, sanear los parámetros generados por el modelo y capturar posibles errores de ejecución. Cuando ocurre un fallo, este se transforma en una observación textual con prefijo `[ERROR]`, que se reintroduce en el historial para que el LLM pueda seguir razonando sobre lo ocurrido en lugar de interrumpir directamente la ejecución.

Tool	Descripción	Backend
`search_flights`	Vuelos ida / ida-vuelta con aerolínea, horarios, escalas, precio y emisiones.	SerpAPI
`search_hotels`	Hoteles por ciudad y rango de fechas con precio, valoración y coordenadas.	RapidAPI (Booking)
`search_airport_transport`	Ruta aeropuerto→hotel por modo de transporte.	API Ninjas + Geoapify
`search_places_of_interest`	POIs por categoría y radio alrededor del destino o alojamiento.	Geoapify Places
`suggest_food_web`	Enlace a una página externa de gastronomía local del destino.	TasteAtlas

Table 1: *Tools* integradas actualmente en el agente.

2. Decisiones de arquitectura

2.1. ReAct vs *function calling* nativo

Se optó por un esquema ReAct en texto libre en lugar de *function calling* nativo. La razón principal es que este enfoque desacopla la lógica del agente de un proveedor o modelo concreto, permitiendo probar distintos LLMs open-source sin depender de una implementación específica de llamadas a funciones. Además, este diseño hace explícito el razonamiento del agente mediante los pasos `Thought`, `Action` y `Observation`, lo que resulta útil tanto para depuración como para mostrar la traza en la interfaz. Por último, facilita la recuperación ante errores: cuando una *tool* falla, el sistema devuelve una observación en formato `[ERROR]...` y permite que el modelo continúe razonando sobre ese fallo en lugar de interrumpir directamente la ejecución.

2.2. Elección del LLM base

El agente utiliza actualmente `Qwen/Qwen2.5-3B-Instruct` como modelo base. Se eligió por ofrecer un compromiso razonable entre tamaño, coste computacional y capacidad para seguir instrucciones estructuradas. En particular, muestra un comportamiento suficientemente estable al generar el formato `Thought/Action/Action Input`, que era un requisito importante para el funcionamiento del bucle ReAct. Además, su tamaño permite ejecutar pruebas tanto en GPU como en CPU, lo que facilita el desarrollo y la experimentación.

2.3. Elección de APIs externas

Para la búsqueda de vuelos se eligió **SerpAPI Google Flights**, ya que proporciona resultados estructurados y buena cobertura. Para el alojamiento se utilizó **Booking.com** a través de **RapidAPI**, que ofrece inventario amplio de hoteles y devuelve información útil como precio, valoración y coordenadas. En la parte de transporte y puntos de interés se empleó **Geoapify**, ya que permite reutilizar la misma API key para geocodificación, rutas y búsqueda de POIs. Finalmente, para resolver coordenadas de aeropuertos a partir de códigos IATA se utilizó **API Ninjas**, evitando mantener una base de datos propia. Como complemento, se añadió una herramienta ligera para enlazar información gastronómica local mediante **TasteAtlas**.

2.4. Parser tolerante y saneado de parámetros

Una dificultad práctica del enfoque ReAct es que el modelo no siempre devuelve un `JSON` limpio. Para hacerlo más robusto, se implementó un extractor balanceado, `_extract_balanced_json`, que recorre el texto contando llaves abiertas y cerradas y tiene en cuenta si se encuentra dentro de una cadena. Esto resulta más fiable que usar expresiones regulares sobre respuestas con estructuras anidadas.

Además, se añadió un paso de saneado de parámetros antes de ejecutar cada herramienta. La función `sanitize_action_input` filtra claves no válidas y normaliza algunos alias frecuentes, reduciendo errores causados por parámetros inventados por el modelo. Junto con esto, el método `_truncate_before_observation` elimina cualquier bloque `Observation:` generado por el LLM en la misma respuesta, forzando así que las observaciones solo provengan de la ejecución real de las tools.

2.5. Gestión de secretos y configuración

Las claves de acceso a las APIs externas se gestionan mediante variables de entorno y `python-dotenv`, evitando que información sensible quede escrita directamente en el código. Este mecanismo se utiliza en todas las tools y también para parámetros de configuración no sensibles, como *timeouts*, idioma o moneda. De este modo, el sistema se mantiene flexible y portable entre distintos entornos de ejecución sin necesidad de modificar el código fuente.

3. Experimentos realizados

3.1. Conjunto de *prompts* probados

Para analizar el comportamiento del agente se definió un pequeño conjunto de *prompts* de prueba, utilizado principalmente con fines exploratorios y cualitativos. Por un lado, se incluyeron consultas con información suficientemente explícita, donde el sistema podía construir directamente una propuesta de viaje bastante completa. Por otro, se probaron consultas más abiertas o ambiguas, pensadas para observar cómo reaccionaba el agente cuando faltaban datos relevantes y debía interpretar mejor la intención del usuario o guiar la interacción.

Prompts utilizados.

- **Con información explícita:**
  - *"Quiero ir de viaje de Madrid a Londres del 8 al 15 de mayo de 2026"*

  - *"Quiero viajar de Madrid a Roma del 10 al 13 de junio de 2026 para 2 personas y me interesan los museos y los restaurantes"*

- **Sin información explícita o más abiertos:**
  - *"Planea una escapada a París con hoteles y lugares culturales"*
  - *"Quiero un viaje gastronómico a Lisboa"*

Con esta selección se buscó comprobar no solo la calidad de las respuestas cuando la petición estaba bien especificada, sino también la capacidad del agente para desenvolverse en escenarios más ambiguos, donde era necesario completar, reinterpretar o refinar la planificación a partir de menos información inicial.

3.2. Modelos comparados

En la parte experimental se compararon varios modelos base con el fin de analizar cuál se adaptaba mejor al comportamiento del agente. En concreto, se consideraron los siguientes modelos:

- `google/gemma-4-E4B-it`
- `meta-llama/Llama-3.2-3B-Instruct`
- `Qwen/Qwen2.5-1.5B-Instruct`
- `Qwen/Qwen2.5-3B-Instruct`

La comparación no se centró únicamente en la calidad textual de la respuesta final, sino también en aspectos más propios del agente, como el seguimiento del formato ReAct, la estabilidad al invocar herramientas externas, la capacidad para recuperarse de errores de parámetros o de API y el tiempo de latencia total necesario para generar una planificación completa. De esta forma, el análisis no solo valora qué modelo produce respuestas más correctas o coherentes, sino también cuál ofrece un mejor equilibrio entre calidad, robustez y tiempo de respuesta.

3.3. Protocolo experimental

Los experimentos se realizaron utilizando el mismo conjunto de *prompts*, la misma arquitectura del agente, el mismo registro de *tools* y la misma lógica de ejecución, de forma que las diferencias observadas dependieran principalmente del modelo base utilizado. La evaluación se diseñó para cubrir dos tipos de situaciones: consultas con información explícita suficiente para generar una propuesta completa desde el principio, y consultas más abiertas, en las que el agente debía desenvolverse con mayor ambigüedad.

La inferencia se llevó a cabo con decodificación determinista, sin muestreo (`do_sample = False`), manteniendo fija la configuración principal del agente. En la versión final se utilizó `max_iterations = 12` y `max_new_tokens = 370`. Aunque el atributo `temperature` aparece definido en la clase, en esta configuración no afecta a la generación al no utilizarse muestreo.

Durante la evaluación se controlaron las mismas condiciones para todos los modelos: mismos *prompts*, mismas *tools* externas, misma configuración del agente y mismo criterio de análisis posterior. A partir de estas ejecuciones se recogieron tanto métricas automáticas como observaciones cualitativas sobre errores de formato, errores de *tool calling*, robustez ante fallos externos y calidad global del plan generado.

4. Métricas de evaluación

Para evaluar el sistema no nos fijamos solo en si la respuesta final parecía buena o no. Intentamos medir tanto la eficiencia del agente durante la ejecución como la calidad objetiva del plan generado. Contemplamos incluir también un modelo de lenguaje (LLM) externo a modo de juez y, aunque llegamos a implementarlo, no conseguimos que ejecutara correctamente y no hemos podido obtener evaluaciones no automáticas sobre los resultados obtenidos.

4.1. Métricas de eficiencia

El primer bloque analiza cómo se comporta el agente dentro del bucle ReAct. En concreto, se mide el coste temporal de la petición, la cantidad de texto generada y el número de pasos y llamadas a herramientas necesarios hasta llegar a la respuesta final.

Latencia media	Tokens	Throughput	Iteraciones ReAct	Tools	Error tools
24.73 s	917	39.2 tok/s	6.8	4.2	18%

La Tabla 4.1 muestra que el agente mantiene un coste de ejecución moderado para este tipo de tarea, con una latencia media de 24.73 segundos por consulta y unas 6.8 iteraciones ReAct por ejecución. En promedio, se generan 917 tokens por sesión, con un throughput de 39.2 tokens por segundo y 4.2 llamadas a herramientas externas. La tasa de error en tools, situada en el 18%, indica que el sistema es funcional, aunque todavía presenta margen de mejora en robustez cuando interactúa con APIs externas.

4.2. Calidad objetiva de la respuesta

El segundo bloque recoge métricas automáticas sobre la respuesta final. Aquí se comprueba si el plan incluye los elementos básicos esperados, si atiende a las fechas y el destino pedidos y, cuando aplica, si se ajusta al presupuesto dado por el usuario.

Además, se añaden dos métricas para medir cómo de buena ha sido la elección del vuelo y del hotel frente al resto de candidatos. A partir de todo ello se obtiene también una puntuación objetiva agregada entre 0 y 1.

Completitud	Fechas	Destino	Presupuesto	Flight rank	Hotel rank	Score agregado
50%	100%	40%	80%	0.80 / 1.00	0.60 / 1.00	0.82 / 1.00

La Tabla 4 muestra un rendimiento intermedio. La completitud media fue del 50%, lo que sugiere que en bastantes casos la planificación no incluía todos los elementos esperados. Sin embargo, el sistema acertó siempre en la interpretación de las fechas solicitadas, cumpliéndolo un 100% de las veces. El destino alcanzó el 40%, sin embargo este número no es representativo, ya que esta métrica mide que la respuesta mencione la ciudad a visitar, y hay veces en las que la respuesta no menciona de nuevo la ciudad explícitamente, aunque sí se refiera a ella correctamente. El presupuesto se respetó en el 80% de los casos. En cuanto a la calidad relativa de las elecciones, el sistema obtuvo una puntuación media de 0.80 en el ranking de vuelos y de 0.60 en el de hoteles, dando lugar a un score de 0.82 sobre 1, lo que indica un comportamiento muy razonable y satisfactorio aunque todavía haya margen para mejorarlo.

5. Análisis de resultados

5.1. Comparativa entre modelos

La comparación entre modelos se realizó de forma principalmente cualitativa, analizando su comportamiento dentro del agente completo más que mediante una evaluación exhaustiva. Se tuvo en cuenta especialmente su capacidad para seguir el formato ReAct, invocar herramientas externas de forma estable, recuperarse de errores intermedios y mantener una latencia razonable.

El modelo más pequeño, `Qwen/Qwen2.5-1.5B-Instruct`, ofrecía respuestas casi inmediatas, pero mostró dificultades para seguir correctamente el *system prompt* y para recuperarse de errores. En cambio, `google/gemma-4-E4B-it` fue el más lento, aunque también el más fiable en términos de razonamiento, estabilidad y gestión de fallos. Los modelos `meta-llama/Llama-3.2-3B-Instruct`

y `Qwen/Qwen2.5-3B-Instruct` presentaron un comportamiento parecido; no obstante, el modelo de Meta requería autenticación adicional en Hugging Face, por lo que finalmente se eligió `Qwen/Qwen2.5-3B-Instruct` como modelo base por ofrecer un compromiso razonable entre calidad, robustez y facilidad de uso.

En las ejecuciones medidas con `Qwen/Qwen2.5-3B-Instruct`, el agente obtuvo una latencia media cercana a 71 segundos por consulta, con unas 7 iteraciones ReAct y 4.6 llamadas a herramientas por petición. La puntuación objetiva media fue aproximadamente 0.68, lo que indica un comportamiento sólido, aunque todavía mejorable en algunos aspectos.

5.2. Ablaciones

Además de la comparación entre modelos, se realizaron varias ablaciones sobre decisiones concretas del diseño del agente, con el objetivo de analizar qué componentes contribuían más a su estabilidad y utilidad práctica.

La primera ablación consistió en eliminar el ejemplo *few-shot* incluido en el *system prompt*. Al hacerlo, se observó que el modelo tendía a seguir peor el formato ReAct y aumentaban los casos en los que la secuencia `Thought/Action/Action Input` no se mantenía de forma consistente. En general, el ejemplo actuaba como una guía importante para estabilizar el comportamiento del agente, especialmente en consultas más largas o cuando había que encadenar varias herramientas.

La segunda ablación consistió en reducir el valor de `max_iterations`. Esta modificación permitió disminuir ligeramente la latencia, pero a costa de que algunas consultas no llegaran a completarse del todo. En particular, cuando la planificación requería varias llamadas consecutivas a herramientas o aparecía algún error intermedio, el agente agotaba antes el número de iteraciones disponibles y generaba respuestas más incompletas.

La tercera ablación se centró en eliminar el saneado previo de parámetros antes de ejecutar las *tools*. En este caso se observó un aumento claro de errores de ejecución, ya que el modelo tendía a generar con más frecuencia claves no válidas, nombres de parámetros incorrectos o categorías que no coincidían exactamente con las esperadas por las APIs externas.

Con estas pruebas se pudo confirmar que la fiabilidad del agente no depende únicamente del modelo base, sino también de varias decisiones de diseño aparentemente pequeñas.

5.3. Tipología de errores

Durante las pruebas se observaron varios patrones de error relativamente claros. En primer lugar, el agente suele comportarse de manera razonable cuando faltan datos esenciales, ya que en muchos casos detecta la ausencia de información obligatoria —por ejemplo, fechas o número de viajeros— y responde pidiendo una aclaración antes de continuar. Este comportamiento fue especialmente útil en consultas abiertas o poco específicas.

Sin embargo, también aparecieron limitaciones importantes en el manejo de ciertas preferencias del usuario. En general, el sistema sí incorpora de forma bastante adecuada los intereses relacionados con los puntos de interés, por ejemplo cuando el usuario menciona museos, restaurantes o lugares culturales. En cambio, funciona peor con preferencias más específicas sobre la optimización del plan, como priorizar precios bajos, elegir alojamientos en una zona concreta o favorecer vuelos directos. En estos casos, la propuesta generada puede seguir siendo razonable a nivel general, pero no ajustarse con precisión a las restricciones o prioridades indicadas por el usuario.

Po último, otro error observado con cierta frecuencia fue la interpretación incorrecta de la información temporal. En particular, cuando el usuario no especifica claramente el año, el sistema tiende a asumir el año 2023, lo que lleva a consultar fechas ya pasadas y provocar fallos posteriores en las herramientas externas.

6. Escalabilidad

Aunque ahora mismo el sistema está centrado en vuelo, hotel, transporte y puntos de interés, la forma en la que está montado permite ampliarlo de una manera sencilla. Como las *tools* están separadas de la lógica principal del agente, añadir nuevas funcionalidades no obliga a rehacer todo el sistema, sino simplemente a incorporar nuevas herramientas.

Aun así, si se quisiera escalar a un uso más real o a muchas consultas, habría que tener en cuenta sobre todo el tiempo de respuesta y la dependencia de APIs externas. En ese caso, tendría sentido optimizar mejor qué herramientas se llaman, reducir iteraciones innecesarias y reutilizar resultados cuando sea posible. En general, la base es escalable, aunque todavía habría margen de mejora en eficiencia.

7. Conclusiones

Este proyecto demuestra que, incluso con una arquitectura relativamente sencilla, es posible construir un agente conversacional útil para la planificación de viajes. A partir de una consulta en lenguaje natural, el sistema genera propuestas que integran vuelos, alojamiento, transporte y puntos de interés, reduciendo considerablemente el esfuerzo de reunir y combinar esa información manualmente. La tasa de error en *tools* se sitúa en torno al 18%, y la mayoría de los fallos se deben a problemas concretos en los parámetros recibidos. Las herramientas más simples, con una única llamada a API, funcionan de forma mucho más estable que las que encadenan varias llamadas usando la salida de una como entrada de la siguiente.

Uno de los aspectos que más esfuerzo nos costó fue el refinamiento del system prompt para que el agente siguiera la estructura ReAct y llamara correctamente a cada *tool*. Las ablaciones confirmaron que modificaciones pequeñas tienen un impacto directo en la estabilidad del agente, y por tanto la fiabilidad del sistema no depende únicamente del modelo base, sino también de decisiones de diseño aparentemente menores.

En cuanto al rendimiento, hemos observado que depende en gran medida del modelo utilizado y de los recursos disponibles. Tras comparar varias alternativas, optamos por `Qwen/Qwen2.5-3B-Instruct` por su equilibrio entre calidad y coste computacional, aunque los tiempos de latencia siguen siendo un factor crítico para la usabilidad. Tuvimos muchos problemas con el servidor DGX, y las APIs externas añaden otra capa de dependencia que condiciona el desarrollo del proyecto (claves, permisos, etcétera). En cuanto a la evaluación, la principal limitación fue la dificultad de encontrar métricas cuantitativas que fueran verdaderamente representativas; incorporar un LLM como juez habría enriquecido considerablemente el análisis, pero no conseguimos que funcionara correctamente.

Como líneas de trabajo futuro (además de arreglar la implementación del *LLM-as-a-judge*), los frentes más relevantes son la incorporación de preferencias del usuario (por ejemplo, con un sistema de prioridades), y el desarrollo de dos modalidades de interfaz complementarias: una conversacional en formato chatbot y otra más guiada, con casillas y cajas a rellenar, que reduzca la ambigüedad en las consultas de entrada y facilite la interacción a un perfil de usuario más amplio.